

0.17. Une lettre qui avance de 5 cases

Bouger, c'est effacer la lettre à sa place, puis la réécrire un peu plus loin — et s'arrêter après le nombre de pas voulu.

1. Ce qu'il faut comprendre

La lettre A avance d'une colonne toutes les **15 images**, soit quatre fois par seconde.

Pour bouger, on **efface** d'abord l'ancienne place avec `effacer(x, 0, 1)`, une case ; puis on ajoute 1 à `x`, et la lettre est réécrite à sa nouvelle place. Sans l'effacement, elle laisserait une traînée de A derrière elle.

Ce qui est nouveau ici : la variable `pas`. Elle range le **nombre de cases** à parcourir : 5. Pour que la lettre aille plus loin ou moins loin, on change ce seul nombre, en haut du programme, sans toucher au reste.

L'arrêt : la condition `if (x < pas)` compare la colonne de la lettre au nombre de pas. Au départ `x` vaut 0 : 0 < 5 est vrai, le chronomètre tourne et la lettre avance. Après le cinquième pas, `x` vaut 5 : 5 < 5 est **faux**, on ne rentre plus dans le bloc, `x` ne change plus... et la lettre reste en colonne 5.

La boucle `while (true)` continue pourtant de tourner : la lettre est simplement réécrite, à chaque image, à la même place.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1389 octets** de cartouche.

1. UNE VARIABLE — PAS

```
uint8_t pas = 5;    // le NOMBRE DE CASES à parcourir : change-le pour aller plus ou moins loin
```

2. UNE VARIABLE — X

```
uint8_t x = 0;      // la colonne de la lettre : elle part de 0 (tout à gauche)
```

3. UNE VARIABLE — IMAGES

```
uint8_t images = 0; // le chronomètre, en images
```

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

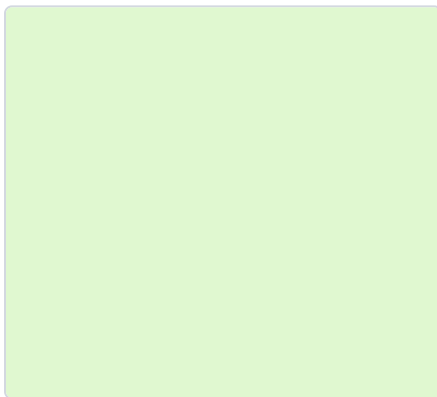
```
int main() {
    while (true) {
        image();                // attend l'image suivante (60 par seconde)

        if (x < pas) {          // tant que la lettre n'a pas fait tous ses pas...
            images++;           // ...le chronomètre tourne
            if (images == 15) { // 4 fois par seconde :
                images = 0;      // le chronomètre repart de zéro
                effacer(x, 0, 1); // 1. efface l'ancienne place
                x++;              // 2. une colonne plus loin
            }
        }                       // x vaut pas (5) : on ne rentre plus ici, elle s'arrête

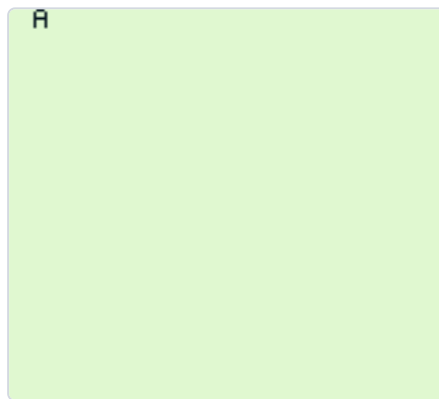
        poser(x, 0, ALPHABET[0]); // 3. la lettre à sa place (nouvelle ou finale)
    }
}
```

3. Ce qui se passe, étape par étape

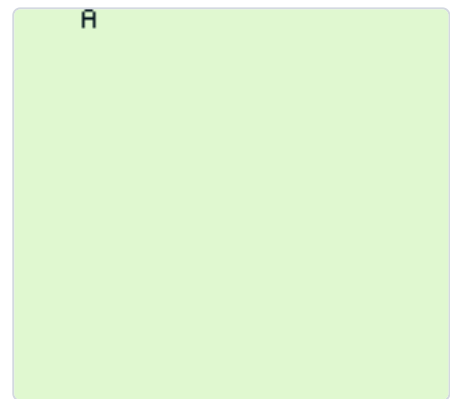
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180

4. Ce qu'on doit voir

Un A qui avance de 5 cases vers la droite, puis s'arrête en colonne 5.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ la lettre a fait 5 pas ($x = 5$)

✓ le A est en colonne 5

✓ un seul A sur la ligne

✓ elle reste arrêtée ($x = 5$)

Ensuite : **0.17.1. Une lettre qui avance de 5 cases — de base, ailleurs** — niveau 0 sur 10. · [gameboy3](#) — écrire une cartouche Game Boy en C++